SPLITR

Interview Master Revision

A concise project guide for placements: architecture, code, DSA, security, scaling, and interview answers.

How to use this PDF
Read the blue callouts first. For every answer, speak in this order: **current design -> example -> validation/algorithm -> production improvement.** Do not memorize paragraphs; understand the facts and explain them naturally.

Project stack

Topic	Remember
Frontend	Next.js 16, React 19, Tailwind CSS, shadcn-style UI, React Hook Form, Zod
Auth	Clerk with Next.js middleware and Convex identity integration
Backend	Convex queries, mutations, schema, reactive subscriptions
Jobs	Inngest cron workflows for reminders and monthly insights
External APIs	Gemini for spending insights; Resend for email
Core algorithm	Directed debt graph + Dinic max flow, inspired by a Splitwise-style published analysis

1. Project pitch and architecture

30-second project pitch

Splitr is a real-time expense-sharing app inspired by Splitwise. Users create personal or group expenses, split them equally, by percentage, or by exact amount, track balances, record settlements, receive payment reminders, and get monthly spending insights. It uses Next.js and React on the frontend, Clerk for authentication, Convex for the reactive backend, Inngest for jobs, Gemini for insights, and Resend for email.

Architecture answer

Splitr has four layers: (1) Next.js/React UI; (2) Clerk authentication and middleware; (3) Convex queries, mutations, schema, and realtime database; (4) Inngest workflows for scheduled email and AI processing.

```
Next.js + React UI
        |
Clerk authentication + protected routes
        |
Convex: queries / mutations / reactive database
        |
Users | Groups | Expenses | Settlements
        |
Inngest jobs -> Gemini insights + Resend emails
```

Create-expense lifecycle

- User enters description, amount, date, payer, participants, group, and split type.
- React Hook Form + Zod validate the form for user experience.
- Frontend calls Convex **createExpense** mutation.
- Backend identifies caller, validates group access and split total, then inserts the expense.
- Reactive Convex queries refresh group views, dashboard data, and balances.

One-line decisions

Topic	Remember
Next.js	Structured React app with routing, layouts, and Clerk integration.
Convex	Reactive reads, backend functions, transactions, and less WebSocket/cache-invalidation work.
Clerk	Delegates security-sensitive authentication and session management.
Inngest	Runs slow, scheduled, retryable work outside normal user requests.

2. Data model and backend code map

Core documents

Topic	Remember
users	name, email, tokenIdentifier, imageUrl. Maps Clerk identity to application user.
groups	name, description, createdBy, embedded members: userId, role, joinedAt.
expenses	description, amount, category, date, payer, splitType, splits, optional groupId, createdBy.
settlements	amount, note, date, payer, receiver, optional groupId, related expense IDs, createdBy.

Key backend functions

Topic	Remember
expenses.createExpense	Writes an expense after group and split-total validation.
expenses.getExpensesBetweenUsers	Builds person-to-person history and net balance.
expenses.getSimplifiedDebts	Creates the group debt graph and returns simplified existing edges.
settlements.createSettlement	Records repayment from payer to receiver.
groups.getGroupExpenses	Returns group records plus member, pairwise, and total balances.
dashboard.*	User balances, total spend, monthly spend, and group summaries.
inngest.*	Outstanding debts and monthly expense data for scheduled jobs.

Indexes to explain

- **expenses.by_group**: fetch one group's expenses.
- **expenses.by_user_and_group**: fetch expenses paid by a user in personal/group context.
- **expenses.by_date**: time-based dashboard and insight queries.
- **users.by_token / by_email**: map authenticated identity to user document.
- Compound-index order matters: an index on (paidByUserId, groupId) supports filtering in that order.

Schema trade-off
Embedded group members and embedded splits make small-group reads simple. At large scale, use **groupMembers** and possibly **expenseSplits** tables with indexes for user-specific queries.

3. Rules, validation, and authorization

Financial invariants - memorize these

- Every amount is positive; every split is non-negative.
- Sum of all split amounts equals the expense amount.
- A user cannot owe themselves.
- A settlement payer and receiver cannot be the same user.
- The sum of all net balances in a group is zero.
- Debt simplification must preserve every user's net balance.
- Retries must not create duplicate expenses or settlements.

Authentication versus authorization

Topic	Remember
Authentication	Who is making the request? Clerk verifies the session; Convex reads the identity through ctx.auth.getUserIdentity().
Authorization	May this specific authenticated user read/write this group, expense, or settlement? Check membership or ownership server-side.
Important rule	Never trust a user ID, payer ID, role, or permission claim sent by the browser.

Defend against direct API calls

A user can bypass the UI and call a public function directly. Therefore every backend mutation must validate: authenticated caller, group membership, payer/participant membership, positive amounts, exact split total, and permissions. Frontend validation improves UX; backend validation protects correctness.

Production hardening answers

- Use integer paise, not floating-point rupees.
- Use idempotency keys for expense and settlement creation.
- Use soft deletion or reversal entries, not destructive history removal.
- Keep secrets server-side; do not pass provider API keys from the client.
- Use audit records: actor, timestamp, reason, before/after values.

4. Ledger and balance calculation

The core mental model

Expenses create obligations. Settlements reduce obligations. Do not overwrite an expense when payment happens; preserve both events and calculate the current balance from the ledger.

Worked example

```
A pays Rs. 1,200 for A, B, C equally

Stored splits: A=400, B=400, C=400
B owes A = 400
C owes A = 400

Net balances: A=+800, B=-400, C=-400
Invariant: +800 - 400 - 400 = 0

If B settles Rs. 200 with A:
A is now owed 600 total; B owes 200; C owes 400
```

Pairwise versus net balance

Topic	Remember
Pairwise	Exact relationship: B owes A Rs. 400. Needed for settlement and debt graph.
Net	Total per user: positive means should receive; negative means should pay.
Why both	A user can be net positive overall while still owing one person and being owed by another.

High-value debugging method

- Reproduce using the smallest set of expenses and settlements.
- Manually calculate every pairwise obligation.
- Compare raw documents, returned balances, and UI display.
- Check duplicate requests, wrong settlement direction, invalid participant, and rounding residue.
- Verify group net sum equals zero.

Strong interview line

I debug balance problems as ledger problems, not display problems. I trace source events, manually recompute obligations, and verify conservation of value.

5. Simplify Debts: graph + Dinic max flow

What your implementation is

Your code implements a debt-simplification approach inspired by the published article *Algorithm behind Splitwise's debt simplification feature*. It is not appropriate to claim it is Splitwise's proprietary production implementation or that it globally minimizes every transaction.

Graph mapping

Topic	Remember
Vertex	A user.
Edge u -> v	u owes v an amount.
Capacity	Debt amount in integer paise conceptually.
Opposite edges	Net first: A->B 500 and B->A 200 becomes A->B 300.
Residual reverse edge	Internal max-flow mechanism to undo/reroute earlier flow; not a displayed debt.

Your exact algorithm steps

- Build a directed debt matrix from expenses and settlements.
- Net opposite debts for every pair of users.
- Save the original directed edges so output never creates a new relationship.
- For each original edge u -> v, build Dinic from remaining current debts and run max flow from u to v.
- Record that flow on u -> v and rebuild remaining graph from residual forward capacities.

Dinic in 30 seconds

BFS builds a **level graph** over positive residual edges. DFS pushes **blocking flow** only from level k to level k+1. Sending flow reduces forward capacity and increases reverse capacity. Repeat until sink is unreachable. Standard worst-case complexity: O(V^2 E); your outer-per-original-edge approach is conservatively O(V^2 E^2).

```
A -> B : 1
A -> C : 5
C -> B : 5

For original A -> B:
max flow A to B = direct 1 + path A->C->B 5 = 6
Output: A -> B : 6 (an existing relationship)
```

Limitations to say honestly

- It does not claim global minimum number of final transactions.
- Explicit directed-cycle cancellation is a separate enhancement.
- Use integer paise; EPSILON only mitigates floating-point noise.

6. Code and DSA cheat sheet

Dinic C++ skeleton - know this cold

```
struct Edge { int to, rev; long long cap; };
vector<vector<Edge>> g;
vector<int> level, ptr;

addEdge(u, v, cap):
  add (v, cap, reverseIndex) to g[u]
  add (u, 0, forwardIndex) to g[v]

bfs(s, t): build levels using cap > 0
dfs(u, t, pushed): use only level[to] = level[u] + 1
maxFlow(s, t): repeat BFS + DFS blocking flow
```

Project-derived coding prompts

Topic	Remember
Net balances	Given [payer, receiver, amount], return each user's net balance. Use array/hash map.
Any settlement	Given net balances, settle using debtor and creditor queues/heaps. Valid plan, not necessarily minimum count.
Minimum settlements	N <= 12: compute net balances, use DFS/backtracking. Heap greedy is not guaranteed optimal.
Equal split	Split totalPaise among N users so sum is exact and share difference <= 1.
Duplicate request	Detect duplicate idempotency keys in a time window using hash map + sliding window.
Debt cycle	Detect directed cycle with DFS colors/recursion stack; cancel minimum edge amount if needed.

Complexity answers

Topic	Remember
Equal split	O(N) time, O(N) output space.
Net balances	O(T) for T transactions, O(N) space.
Top K spenders	Hash map totals + min heap: O(T + U log K).
Cycle detection	DFS: O(V + E).
Dinic	General O(V^2 E); your repeated edge flow approach conservatively O(V^2 E^2).

7. React, Next.js, and Convex

Realtime explanation

Convex queries are reactive subscriptions. When a mutation changes data read by a subscribed query, Convex re-runs the query and pushes updated results to the client. This avoids manual polling, custom WebSockets, and cache invalidation for common flows.

Frontend details to know

Topic	Remember
Client components	Expense form, split selector, dialogs, and interactive views use hooks/state and run in the browser.
React Hook Form	Manages form registration, errors, submit state, reset, and field watching.
Zod	Structured client validation; backend still repeats critical validation.
SplitSelector	Local dynamic state for equal, percentage, exact splits; validate total before submit.
skip	Avoids a Convex query until conditional UI state and arguments are ready.
Custom hooks	Standardize data, loading, error, mutation behavior; wrappers should remain thin.

React pitfalls

- Do not put setTimeout or other side effects in render; use useEffect with cleanup.
- Arrays/functions in effect dependencies are compared by reference; unstable references can cause loops.
- Separate loading from empty state to prevent empty-page flashes.
- Disable submit for UX, but use idempotency on backend for correctness.
- Use accessible labels, keyboard navigation, focus management, and non-color-only errors.

Convex function distinctions

Topic	Remember
Query	Reads data; reactive for UI. Keep it bounded and indexed.
Mutation	Changes application data transactionally.
Action	Calls external services such as email/AI; not a DB transaction.
Internal function	Backend-only helper; reduces public attack surface.

8. Background jobs, Gemini, and email

Workflow

```
Cron trigger
  -> find eligible users
  -> fetch limited recent/aggregated spending data
  -> Gemini generates insight
  -> validate/sanitize output
  -> Resend sends email
  -> store delivery outcome
```

Why Inngest

Background work must not block expense creation. Inngest provides cron triggers, named durable steps, retries, observability, and isolation of per-user failures.

Reliability checklist

- Use an email-delivery record keyed by user + message type + period.
- Use idempotency to prevent duplicate messages when a provider times out.
- Re-check latest balance just before sending a reminder.
- At scale: scheduler pages eligible users, emits one event per user, workers process with controlled concurrency.
- Keep AI/email failure separate from the financial ledger; core expense flows must still work.

AI security and privacy

Topic	Remember
Data minimization	Prefer category totals and trends over raw descriptions or unnecessary identity data.
Prompt injection	Treat descriptions as untrusted data; model must not have access to extra tools/data.
Model output	Never trust raw HTML. Sanitize with allowlist or request structured JSON and render with trusted React.
Cost	Limit users, frequency, input size, output size; cache one insight per period.
Secrets	Gemini and Resend keys remain server-side environment variables only.

9. Scaling and production answers

If dashboard is slow at 100,000+ expenses

- Measure query latency, documents read, bytes returned, and mutation retries first.
- Replace unbounded collect/filter logic with targeted indexes and cursor pagination.
- Keep raw ledger as source of truth; maintain transactionally updated derived balance/monthly aggregates for fast reads.
- Separate summary queries from paginated history queries.

Common redesigns

Topic	Remember
Many groups per user	Create groupMembers table indexed by userId and groupId.
Many split searches	Create expenseSplits table indexed by userId and expenseId.
Large history	Cursor pagination: newest page first, fetch next cursor on scroll.
Hot group writes	Avoid one high-churn aggregate doc; use immutable events and smaller per-member/pair aggregates.
Recurring expenses	Store templates and create immutable monthly occurrence with idempotency key.
Multiple currencies	Store amount in minor units plus currency. Do not merge currencies without explicit locked exchange rate.

The answer framework for any scenario

Use this every time

1. State current behavior. **2.** Identify risk/edge case. **3.** Give backend/data/algorithm fix. **4.** State trade-off.

Example: 'I would measure first, then replace full collection scans with an index and cursor pagination. I would precompute current balances but retain raw events as source of truth.'

10. High-frequency interview questions

Project and architecture

- Tell me about Splitr. What problem does it solve?
- Walk through the create-expense request from form to updated balance.
- Why Next.js + Convex rather than React + Express + PostgreSQL?
- What does real-time mean here and how is it different from polling?
- Why separate expenses from settlements?

Database and correctness

- Explain your schema, indexes, and the trade-off of embedded members/splits.
- How do you prove group balance is correct?
- How do you prevent duplicate settlement after a timeout?
- How do you split Rs. 100 among three users exactly?
- What happens if an old settled expense is edited?

DSA

- Explain graph modelling and debt simplification.
- Why Dinic? Explain level graph, residual graph, reverse edge, blocking flow.
- Code max flow / net balances / minimum settlements.
- Does your approach minimize all transactions? What is its limitation?

Security and scale

- A user guesses a group ID. How do you stop data leakage?
- A client fakes a payer or participant. What must the backend validate?
- The dashboard takes 12 seconds with 100,000 expenses. Redesign it.
- How do you secure Gemini, Resend, and AI-generated HTML?

Honest production-improvement answer

> *My first hardening pass would store money as integer paise, enforce all participant and authorization rules server-side, use idempotency keys, move high-volume membership and split relationships into indexed tables, paginate history, sanitize AI output, keep secrets server-side, and add immutable audit events plus ledger-invariant tests.*

Final reminder
Do not claim that a prototype is production-perfect. Strong candidates explain the current design clearly, name the real limitation, and propose a concrete safe improvement.